How to Write Effective Skill Markdown Files for Coding Agents

Skill markdown files are specialized instructions provided to a coding agent (like Claude Code or Pi) to teach it a specific domain, a set of coding standards, or a complex workflow. When written well, they significantly reduce hallucinations and increase the success rate of complex tasks.

1. Core Purpose

A skill file should not just be a "manual"; it should be a **set of behavioral guidelines**. Instead of just saying *what* something is, focus on *how* the agent should act when it encounters a specific scenario.

2. Essential Structure

A high-quality skill file should typically include the following sections:

A. Purpose & Scope

Clearly define what this skill is for and, more importantly, **when it should be applied**. - **Bad:** "This is a guide for React." - **Good:** "Use this skill when implementing new UI components using React 18+ and Tailwind CSS in the `/src/components` directory."

B. The "Golden Rules" (Constraints)

List absolute requirements and prohibitions. Use strong language. - **Must:** "Always use functional components with TypeScript interfaces for props." - **Must Not:** "Never use inline styles; always use Tailwind utility classes."

C. Step-by-Step Workflow

Break down a complex task into a logical sequence of actions the agent should take. 1. **Analyze:** Check existing patterns in `X` file. 2. **Plan:** Write a brief plan in the chat before editing. 3. **Implement:** Apply changes following the patterns found. 4. **Verify:** Run `npm test` to ensure no regressions.

D. Examples (The "Few-Shot" Approach)

Examples are the most powerful part of a skill file. Provide "Before" and "After" or "Wrong" and "Right" code snippets. - **Wrong:** `const data = await fetch(...)` (missing error handling) - **Right:** `try { const data = await fetch(...); ... } catch (e) { ... }` (proper error handling)

E. Verification & Validation

Tell the agent how to prove it did the job correctly. - "The task is complete only if: - All new functions have JSDoc comments. - The CI pipeline passes. - The updated documentation is reflected in `README.md`."

---

3. Writing Tips for Better Agent Performance

Be Explicit, Not Implicit

Agents struggle with "common sense." If you want a variable named in `camelCase`, say it. If you want a specific import order, list it.

Use "Trigger Phrases"

Use keywords that the agent is likely to see in the codebase or user prompts. This helps the agent associate the skill file with the current task.

Keep it Modular

Instead of one giant `KNOWLEDGE.md`, create smaller, focused skill files: - `skill_testing_standards.md` - `skill_api_design.md` - `skill_deployment_workflow.md`

Avoid Verbosity

While being explicit is good, avoid "fluff." Use bullet points and checklists. The agent has a context window; make every token count.

---

4. Example Template

```
# Skill: [Skill Name]

## Context
Apply this skill when [Specific Trigger/Scenario].

## Standards & Constraints
- [Constraint 1]
- [Constraint 2]
- **Prohibited:** [What to avoid]

## Workflow
1. [Step 1]
2. [Step 2]
3. [Step 3]
```

```
## Code Examples
###   Incorrect
```[lang]
// code here
```

**Correct**

```
// code here
```

Success Criteria

- ☐ Checklist item 1
- ☐ Checklist item 2 “‘

5. References

This guide is based on industry-standard prompt engineering principles:

- Anthropic Prompt Engineering Guide - Best practices for structuring prompts for Claude models.
- OpenAI Prompt Engineering Guide - Fundamental strategies for guiding Large Language Models (LLMs).
- Few-Shot Prompting - Learn more about the effectiveness of providing examples to models.
```